

Detección de Fraude en Tarjetas de Crédito

Mediante Algoritmos de Ensamble

Arturo Ayala Hernández
Eugenio Hernandez Aranda
Santiago Nava Pulido

Objetivos del Estudio



Objetivo General

Desarrollar un modelo predictivo basado en algoritmos de ensamble capaz de clasificar con máxima precisión las transacciones legítimas y sobre todo fraudulentas



Objetivos Específicos

- Preparar datos.
- Implementar modelos
- Evaluar desempeño

Marco Teórico: ENSAMBLE DE MODELOS

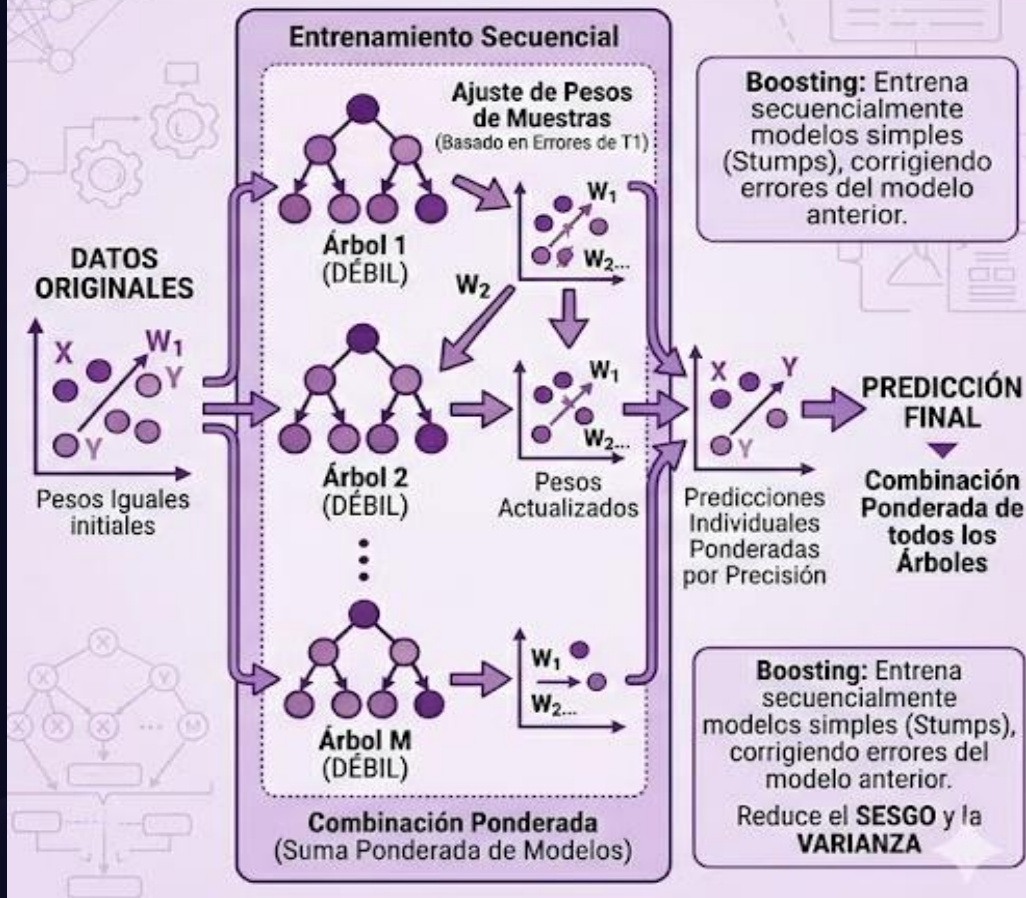
BOOSTING

Método secuencial $\rightarrow$ suma ponderada de resultados

Se evalúan los errores del modelo anterior. Los datos que fueron mal clasificados o tienen mayores errores reciben un mayor peso o importancia para el siguiente paso.

Reducir el sesgo y la varianza

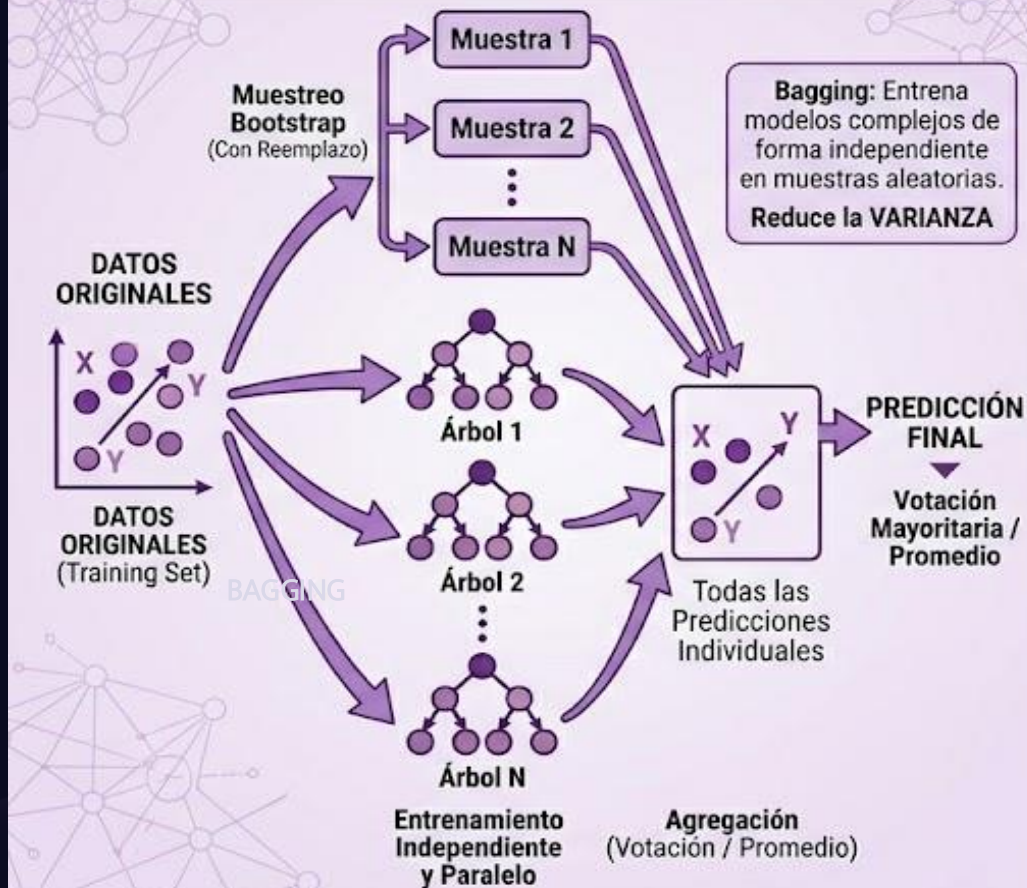
2. PROCEDIMIENTO BOOSTING (e.g., AdaBoost)



BAGGING

- Se crean múltiples subconjuntos de datos (Muestras Bootstrap) mediante muestreo aleatorio con reemplazo.
- Reduce la Varianza
- Paralelo

1. PROCEDIMIENTO BAGGING (Bootstrap Aggregating)



Credit Card Fraud Detection Dataset



- **Datos:** 10,000 transacciones -> 10 var.
- **Clasificación:** Operaciones etiquetadas como fraude (1) o legítimas (0).
- **Objetivo:** Encontrar patrones no lineales de fraude.
- **Resultado Esperado:** Un modelo predictivo de alta precisión, capaz de lidiar con datos desbalanceados.



EDA

Alerts

`foreign_transaction` is highly imbalanced (53.8%)

Imbalance

`location_mismatch` is highly imbalanced (57.8%)

Imbalance

`is_fraud` is highly imbalanced (88.7%)

Imbalance

`transaction_id` is uniformly distributed

Uniform

`transaction_id` has unique values

Unique

`velocity_last_24h` has 1378 (13.8%) zeros

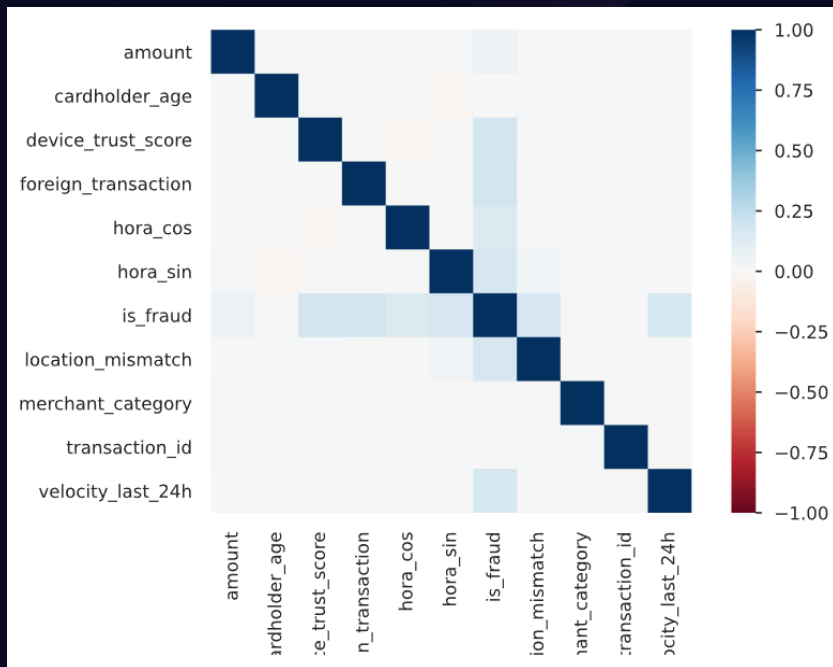
Zeros

`hora_sin` has 417 (4.2%) zeros

Zeros

```
#from pandas_profiling import ProfileReport
```

Grafica de Correlación



is_fraud

Categorical

Imbalance

Distinct

2

Distinct (%)

< 0.1%

Missing

0

0

9849

1

151

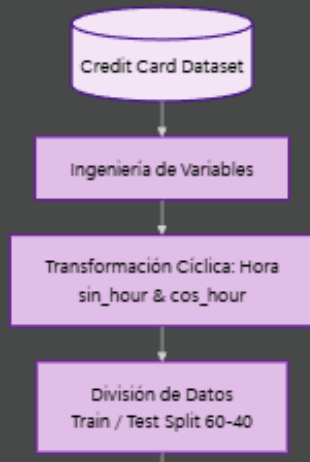
ANALISIS DE COLINEALIDAD (VIF)

Reporte de Factor de Inflación de la Varianza (VIF)

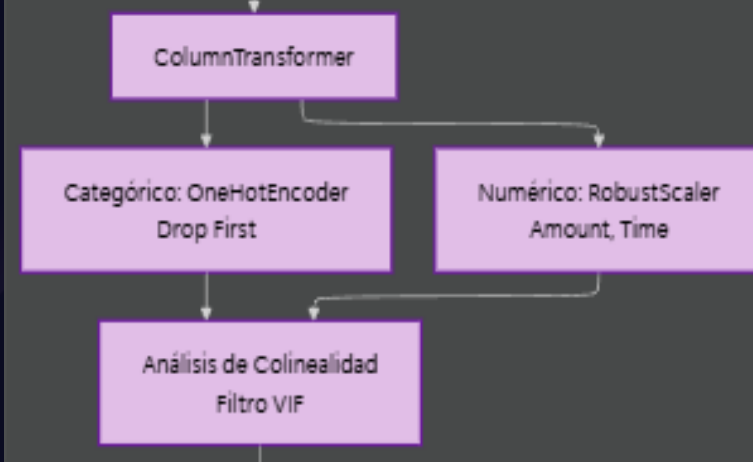
feature	VIF
bin__foreign_transaction	1.0886
bin__location_mismatch	1.0766
num__amount	1.0750
cat__merchant_category_Food	1.0633
cat__merchant_category_Electronics	1.0577
cat__merchant_category_Travel	1.0572
cat__merchant_category_Grocery	1.0521
num__cardholder_age	1.0021
bin__hora_cos	1.0016
num__device_trust_score	1.0015
bin__hora_sin	1.0013
num__velocity_last_24h	1.0004

*statsmodels.stats.outliers_influence import variance_inflation_factor

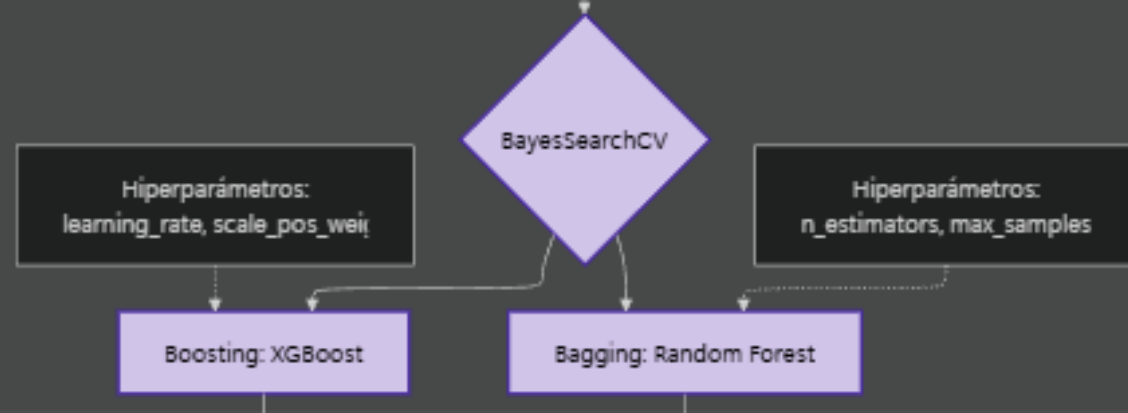
Fase 1: Preparación de Datos



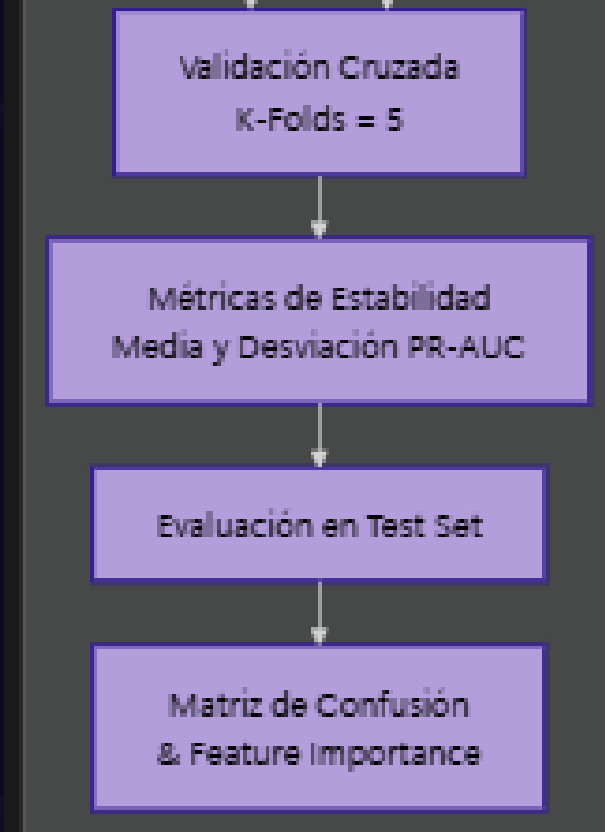
Fase 2: Pipeline de Transform



Fase 3: Entrenamiento y Tuning



Fase 4: Validación y Resultados



BayesSearch(a)

```
'MODEL__n_estimators': (50, 100), #numero de arboles  
'MODEL__max_samples': (0.5, 1.0), #cantidad de datos que realizara el bootstrap  
'MODEL__estimator__max_depth': (2, 20), #profundidad de los arboles  
'MODEL__estimator__class_weight': ['balanced', 'balanced_subsample'] #pesos en base a los conjuntos o por cada subconjunto
```

BayesSearch(b)

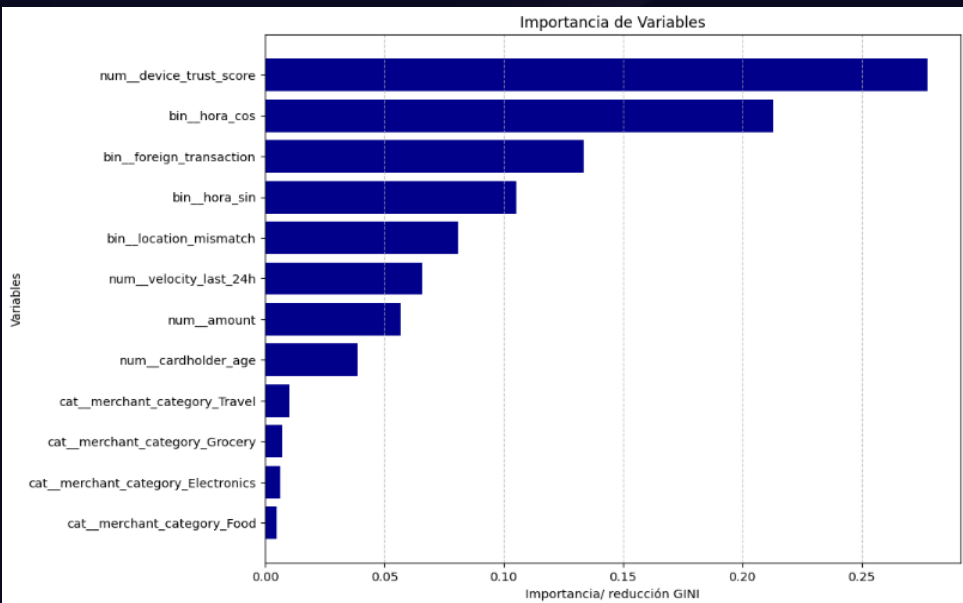
```
'MODEL__n_estimators': (50, 300), # Cantidad de iteraciones de boosting  
'MODEL__learning_rate': (0.01, 0.2, 'log-uniform'), # Ritmo de aprendizaje  
'MODEL__max_depth': (3, 7), # Árboles poco profundos para evitar overfitting  
'MODEL__subsample': (0.6, 0.9), # Porcentaje de datos por árbol para dar varianza  
'MODEL__colsample_bytree': (0.5, 0.8), # Fracción de columnas (features) por árbol  
'MODEL__scale_pos_weight': (50, 150), # Penalización al error en fraude (ajustar según desbalance)  
'MODEL__gamma': (0, 5), # Regularización: reducción mínima de pérdida para dividir  
'MODEL__reg_alpha': (1e-5, 10, 'log-uniform') # Regularización L1
```

SCORING Average Precision – PR-AUC

PRECISIÓN + RECALL,
ideal para data desbalanceada

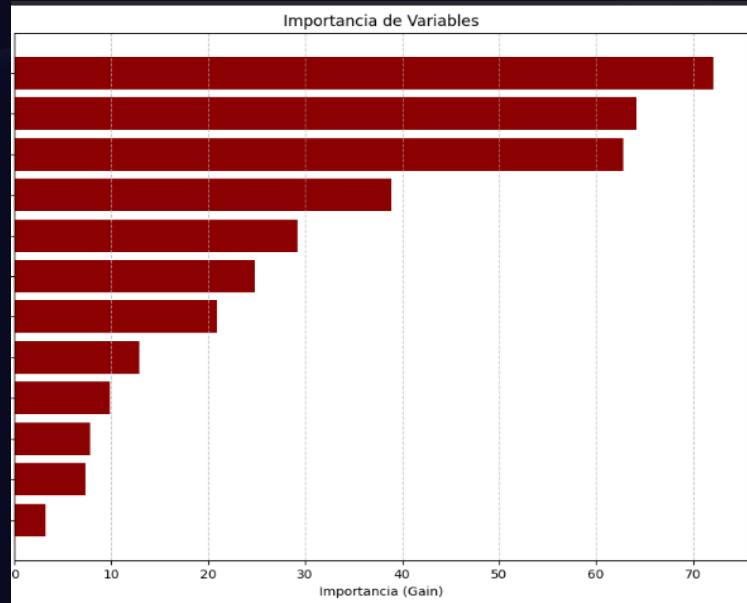
Árboles de Decisión

Mejora relativa que aporta cada variable



RandomForestClassifier + Bagging

Modelo predictivo que utiliza estructura de árbol para estimar valores continuos.

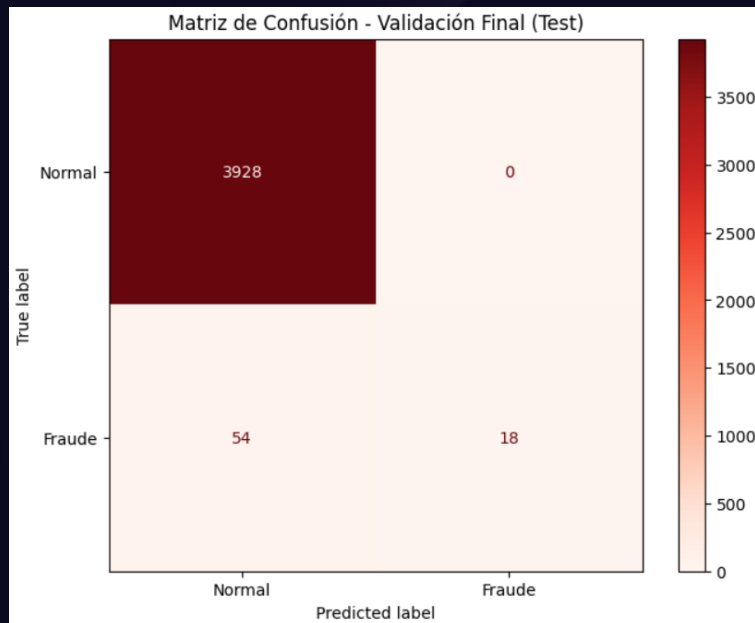


XGBoost

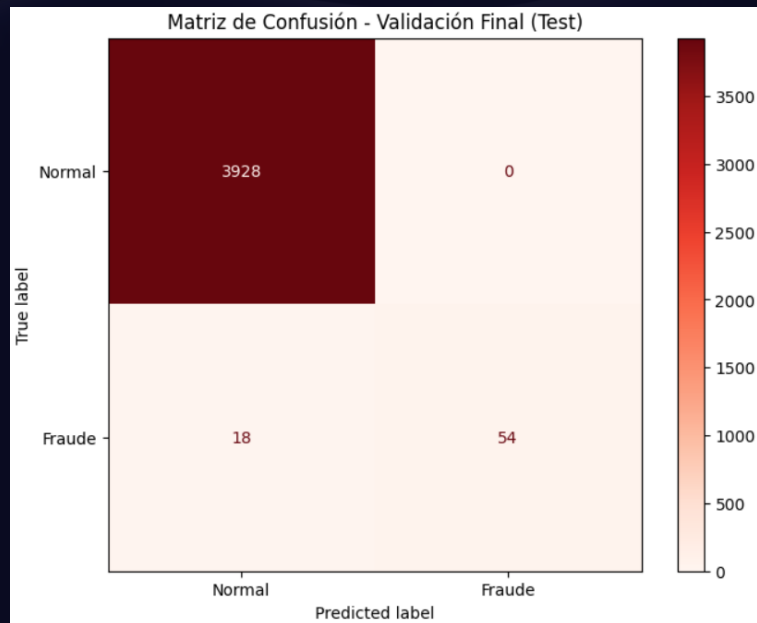
Divide datos en categorías mediante criterios de decisión en nodos.

Matrices de Confusion

Random Forest



BOOSTING



MÉTRICAS A

Modelo	Fase	Métrica	Valor (Promedio / Resultado)
Bagging	Entrenamiento (CV)	Precision	0.80000 ± .4
Bagging	Validación Final (Test)	Precision	1.00000
Bagging	Validación Final (Test)	Recall	0.25000
Bagging	Validación Final (Test)	F1 Score	0.40000
Bagging	Validación Final (Test)	Accuracy	0.98650

Notamos que el modelos posee excesiva varianza la cual lo vuelve poco confiable, sin embargo el Precision promedio es bueno. Aunque no vemos reflejado la misión del Optimizer.

Distinguiendo las decisiones dentro de la matriz de confusión y los métricos, nuestro modelo aunque posee un precisión perfecto no detecto prácticamente 75% de los fraudes los cual no empata con el objetivo que buscábamos.

MÉTRICAS B

Modelo	Fase	Métrica	Valor (Promedio / Resultado)
XGBoost	Entrenamiento (CV)	Precision	0.9714 ± .034
XGBoost	Validación Final (Test)	Precision	1.00000
XGBoost	Validación Final (Test)	Recall	0.75000
XGBoost	Validación Final (Test)	F1 Score	0.85710
XGBoost	Validación Final (Test)	Accuracy	0.99550

Al contrario de bagging, boosting consiguió de forma muy consistente (desviación +- .034) teniendo un precision de .97, lo cual nos habla de un modelo muy seguro de sus decisiones.

Poseemos un modelo muy confiable con precision prácticamente perfecto y además identificando el 75% de los fraudes que sucedieron en este fragmento del dataset.

Conclusiones y Referencias

"Los resultados concluyen que XGBoost es la arquitectura óptima para la detección confiable de fraudes crediticios. Su desempeño superó consistentemente al de Bagging, logrando una precisión superior con una varianza mínima en las pruebas de validación cruzada.

Por el contrario, se desaconseja el uso de Bagging para esta problemática específica, debido a su elevada inestabilidad y a su limitada capacidad para identificar correctamente los patrones de fraude en entornos de alta complejidad."

Referencias

Elserty, A. (2020). Credit Card Fraud Detection Dataset [Conjunto de datos]. Kaggle.
<https://www.kaggle.com/code/abdelazizelserty/credit-card-fraud-detection-dataset>

James, G., Witten, D., Hastie, T., Tibshirani, R., & Taylor, J. (2023).
An introduction to statistical learning: With applications in Python. Springer.